

YOUVERSE TECH REPORT

1. Introduction / Problem Statement

Modern behavioral and wellness applications rely on continuous environmental context to model user patterns. Existing solutions either depend on smartphone sensors (limited accuracy, high power) or expensive commercial IoT hubs. Youverse addresses this gap with a low-cost, dedicated ESP32-based sensor node that collects environmental and behavioral proxy signals in real time.

Problem: Individuals lack a simple, always-on hardware node that can capture light, temperature, humidity, motion, and ambient noise levels, display them locally for transparency, and stream structured data to a backend for AI-driven behavioral modeling (e.g., “Focused You”, “Active You”, “Burned-Out You”).

Solution: An ESP32-WROOM-32 sensor node that periodically samples five signals, renders live readings on an SSD1306 OLED, and posts a standardized JSON payload over HTTPS to a Railway-hosted backend. The system is fully designed in KiCad (schematic + PCB) and validated end-to-end in Wokwi simulation.

Target users: Students, makers, and early adopters of personal analytics / quantified-self tools who want an open, extensible hardware foundation rather than closed commercial wearables.

2. System Overview

Youverse is a single-board environmental/behavioral sensor node. Data flows as

follows: **Sensors** → **ESP32** → **OLED (local display) + Wi-Fi (HTTPS POST)** →

Backend / Supabase

Key features:

- Five simultaneous sensor channels: temperature, humidity, light, motion, noise
- Local OLED feedback showing live values and transmission status
- ISO-8601 timestamps via NTP for accurate time-series data
- HTTPS POST with retry logic (up to 3 attempts) to a defined Sensor Data Contract
- Fully simulated and verified on Wokwi; physical PCB design underway
- Designed for future low-power improvements (Deep Sleep capable)

High-level block diagram (textual):

```
[DHT22]■■■  
[LDR]■■■■■  
[PIR]■■■■■■■■■ ESP32-WROOM-32 ■■■ SSD1306 OLED (I2C)  
[POT]■■■■■ ■  
■■■■ Wi-Fi / HTTPS ■■■ Backend API
```

3. Hardware Design

3.1 Component Selection

Components were chosen for low cost, wide availability, and compatibility with both Wokwi simulation and future physical assembly. The ESP32 provides integrated Wi-Fi, sufficient ADC channels, and dual-core performance while remaining power-manageable.

1	ESP32-WROOM-32 / DevKit V1	1	Main MCU + Wi-Fi
2	DHT22 (or DHT11)	1	Temperature + Humidity
3	LDR (Photoresistor)	1	Ambient light level
4	PIR HC-SR501	1	Motion detection
5	10 kΩ Potentiometer	1	Manual noise input (demo)
6	SSD1306 OLED 0.91" I2C	1	Local display (128×64)

7	10 kΩ Resistor	2	DHT pull-up + LDR divider
8	AP2112K-3.3 LDO	1	3.3 V regulation (PCB)
9	10 μF Capacitors	2	Power decoupling
10	USB Micro-B connector	1	Power + programming

Table 1 — Bill of Materials (simulation + PCB design)

3.2 Pin Mapping

DHT22	DATA	GPIO4	Digital, 10 kΩ pull-up
LDR	Analog Out	GPIO34	ADC1_CH6 (input only)
PIR	OUT	GPIO27	Digital input
Potentiometer	Wiper	GPIO35	ADC1_CH7 (noise proxy)
SSD1306 OLED	SDA	GPIO21	I2C data
SSD1306 OLED	SCL	GPIO22	I2C clock

Table 2 — Pin assignment

3.3 Schematic & Power Design

The schematic (KiCad) centers on the ESP32-WROOM-32 module. Power is supplied via USB Micro-B and regulated to 3.3 V by an AP2112K-3.3 LDO with 10 μF input/output decoupling capacitors. The DHT22 DATA line uses a 10 kΩ pull-up to 3.3 V. The LDR forms a simple voltage divider. PIR and potentiometer are directly connected to digital and ADC pins respectively. The OLED is driven over I2C (address 0x3C).

ERC report: 32 messages — all expected unused pins (ESP32 SPI-flash pins, USB D+/D-/ID/Shield,

unused GPIOs). Zero functional connectivity errors. Schematic is production-ready for the current feature set.

3.4 PCB Design Status

A custom PCB layout has been started in KiCad. Current DRC status: 0 design-rule violations. A few unconnected pads remain (mainly USB shield and power-zone stitching) and will be resolved before Gerber export. The board includes a keep-out zone for the ESP32 antenna, proper 3.3 V plane, and footprints for all sensors and the OLED. Physical assembly is planned post-hackathon once components are sourced.

4. Firmware Design

4.1 Platform & Tools

Firmware is written in Arduino C++ for the ESP32 core (arduino-esp32 3.x). Build is performed with arduino-cli (FQBN: esp32:esp32:esp32). Required libraries: DHT sensor library, Adafruit SSD1306, Adafruit GFX, WiFi, HTTPClient, WiFiClientSecure.

4.2 Overall Architecture

The firmware follows a classic `setup()` / `loop()` structure with clear separation of concerns:

- `setup()` — Serial, pin modes, DHT begin, OLED init, Wi-Fi connect, NTP time sync
- `loop()` — Sensor read → value mapping / fallback simulation → JSON build → OLED update → HTTPS POST → OLED status update → 5 s delay

4.3 Sensor Acquisition & Processing

Every 5 seconds the node samples:

- DHT22 → temperature (°C) and humidity (%)
- LDR (GPIO34) → `analogRead` → mapped 0–100 light level
- PIR (GPIO27) → `digitalRead` → 0/1 motion flag

- Potentiometer (GPIO35) → `analogRead` → mapped 0–100 noise level

If DHT readings return NaN (common in pure simulation), realistic fallback values are generated so the pipeline remains continuous for backend testing. Light and noise also receive small random offsets when near-zero to avoid flat-line data during demos.

4.4 Networking & Data Contract

Wi-Fi credentials target the Wokwi guest network for simulation. After connection, NTP (pool.ntp.org) synchronizes the system clock. Each payload is a compact JSON object matching the agreed Sensor Data Contract:

```
{
  "device_id": "esp32-sim-001",
  "timestamp": "2026-08-16T12:00:00Z",
  "light": 77.7,
```

```

    "temperature": 25.0,
    "humidity": 50.0,
    "motion": 1,
    "noise": 30.0
}

```

Transmission uses `WiFiClientSecure` (`setInsecure` for staging) + `HTTPClient` POST to `https://youverse-stag-3.up.railway.app/api/sensors/ingest`. Up to three retries with 1 s delay are performed on failure. The OLED shows “Sending...” before the request and “Sent OK” (or remains on previous state) after.

4.5 Display

The SSD1306 (I2C 0x3C) shows a simple status screen: title, temperature, humidity, light, motion, and current transmission status. This provides immediate local feedback without affecting the data stream.

4.6 Power Considerations (Future)

The current firmware runs continuously with a 5-second sampling interval (suitable for simulation and demo). For battery operation the natural next step is to replace the fixed delay with ESP32 Deep Sleep (timer wake-up), power-gate or sleep the OLED, and keep Wi-Fi off except during the short transmit window. Deep Sleep currents of 10–150 μ A are achievable, enabling multi-day operation on a small LiPo cell.

5. Technical Specifications

MCU	ESP32-WROOM-32 (dual-core Xtensa, up to 240 MHz)
Operating Voltage	3.3 V (USB 5 V $\rightarrow$ AP2112K LDO)
Temperature Range (DHT22)	−40 °C to +80 °C ($\pm$ 0.5 °C)
Humidity Range (DHT22)	0–100 % RH ($\pm$ 2–5 %)
Light Sensing	LDR + ADC, mapped 0–100
Motion	PIR digital (GPIO27)
Noise Proxy	10 k Ω potentiometer $\rightarrow$ ADC (GPIO35)
Display	SSD1306 128×64 OLED, I2C
Connectivity	Wi-Fi 802.11 b/g/n, HTTPS POST
Sampling Interval	5 seconds (configurable)
Timestamp	ISO-8601 UTC via NTP
Firmware Size	~1.08 MB application binary
Simulation Platform	Wokwi (ESP32 + sensors + OLED)
Design Tools	KiCad 10.x (schematic + PCB)

Table 3 — Technical specifications

6. Testing & Results

The complete pipeline has been verified live:

- Wokwi simulation loads the compiled firmware and drives all sensors + OLED
- Sensor values appear correctly on the OLED and in Serial output
- JSON payloads match the agreed Sensor Data Contract
- HTTPS POST reaches the Railway staging endpoint and is stored in Supabase
- Retry logic exercises correctly on transient failures
- NTP timestamps are valid ISO-8601 strings
- Schematic ERC is clean of functional errors; PCB DRC reports 0 violations

Limitations of the current stage: the system is simulation-first; physical PCB has not yet been fabricated. Noise is currently a manual potentiometer rather than a true microphone/ADC noise sensor. Continuous 5 s sampling is not power-optimized for battery use.

7. Challenges & Future Improvements

Challenges encountered:

- Wokwi DHT sensor occasionally returns NaN → solved with realistic fallback

generation • HTTPS certificate handling on ESP32 → temporary `setInsecure()` for staging

- Balancing schematic cleanliness with unused ESP32 pins (expected ERC noise)
- Keeping PCB antenna keep-out and power integrity while fitting all parts

Planned / recommended next steps:

- Complete PCB routing, resolve remaining unconnected pads, generate Gerbers
- Physical prototype assembly and validation against simulation
- Replace potentiometer noise proxy with a proper analog microphone + amplifier or digital MEMS mic
- Implement ESP32 Deep Sleep + timer wake-up for multi-day battery life
- Add OTA firmware update capability
- Optional local data buffering (SPIFFS / LittleFS) when Wi-Fi is unavailable
- Enclosure design and mounting for desktop / wall use

8. Conclusion

Youverse delivers a complete, working embedded sensor node that captures five environmental/behavioral signals, provides local visual feedback, and streams structured data to a backend for higher-level AI modeling. The project demonstrates the full embedded workflow expected by the Reverie Hacks Embedded Systems track: problem identification, component selection, schematic design (ERC-clean), PCB layout (in progress), firmware development, and end-to-end verification via simulation.

All required deliverables — file directory (schematics, BOM, simulation, firmware), firmware with algorithm flowchart, demo capability, and this technical report — are satisfied. The architecture is intentionally simple, documented, and ready for both continued development and physical realization after the hackathon.

Appendix A — Firmware Algorithm Summary

A dedicated Firmware Algorithm Flowchart PDF accompanies this report. The high-level sequence is:

1. Power-on / Reset → initialize Serial, pins, DHT, OLED
2. Connect Wi-Fi (Wokwi-GUEST) and wait for association
3. Synchronize time via NTP
4. Enter main loop:

- a. Read DHT (temp, humidity), LDR, PIR, potentiometer
- b. Map analog values; apply fallback simulation if needed
- c. Build ISO timestamp and JSON payload
- d. Update OLED (“Sending...”)
- e. HTTPS POST with up to 3 retries
- f. Update OLED (“Sent OK”)
- g. Delay 5 seconds → repeat